OPEN BOOK / ฉบับลงมือทำ

ชีททำตามทีละข้อ

RESTful API ด้วย Express.js

อ้างอิง exam-spec ชุด 67130500001–67130500004 • ใช้ร่วมกับ Windows / PowerShell / WebStorm • เริ่มตั้งแต่เปิดโฟลเดอร์จนส่ง ZIP

> วิธีใช้: ทำหน้า 2–8 ก่อนหนึ่งครั้ง แล้วทำข้อสอบตามหน้า 9–23 ทีละข้อ ทุกครั้งที่เพิ่มโค้ดให้บันทึกไฟล์ หยุด Server ด้วย Ctrl+C แล้วรัน npm start ใหม่

เปิดหาเรื่อง	หน้า
ข้อมูลนักศึกษา + สิ่งที่โจทย์กำหนด / ไม่ได้กำหนด	2
เปิดโฟลเดอร์จริง ติดตั้ง npm และนำเข้าฐานข้อมูล	3–4
ไฟล์กลาง + โครง Route ที่ต้องสร้างก่อน	5–7
ข้อ 2.1 Student Info: app.js และการทดสอบ	8–9
ข้อ 2.2 Brands: ทั้งหมด / ตาม ID	10 / 11
ข้อ 2.3.1 Products: Summary DTO + Pagination	12–14
ข้อ 2.3.2 Products: Detail DTO + JOIN	15–17
ข้อ 2.3.3 POST: เพิ่มสินค้า + SKU ซ้ำ	18–20
ข้อ 2.3.4 PUT: แก้ไขสินค้า	21–22
ข้อ 2.3.5 DELETE: ลบสินค้า	23
ประกอบไฟล์ ตรวจงาน ทดสอบ และส่งข้อสอบ	24–27
ตารางแก้ Error / แนวโจทย์เสริม / สรุปจำเป็น	28–31
แหล่งอ้างอิงและขอบเขตการตรวจโค้ด	32

ในแต่ละข้อมีอะไร

เป้าหมายตาม spec → ไฟล์ที่ต้องแก้ → ตำแหน่งวางโค้ด → โค้ดตัวอย่างครบ → อธิบายการทำงาน → วิธีทดสอบ → จุดพลาดที่ต้องตรวจ

> โค้ดในชีทเป็นตัวอย่างเฉลยที่จัดทำเพิ่ม ไม่ใช่โค้ดเฉลยจากอาจารย์ ชื่อ table / column ต้องตรวจจาก SQL จริงก่อนใช้ และใช้เอกสารนี้ในห้องสอบตามกติกาของผู้สอน

00 / ขอบเขตและข้อมูลประจำตัว

ก่อนเริ่ม: เลือก spec ของตัวเอง

ทั้งสี่ชุดมี Endpoint และเงื่อนไขหลักตรงกัน จึงใช้วิธีทำชุดเดียวกันได้ เปลี่ยนเฉพาะข้อมูลนักศึกษา Base Path คำสั่งตรวจ และชื่อ ZIP ให้ตรงชุดที่ได้รับ

STUDENT_ID	STUDENT_NAME	SECRET_KEY
67130500001	Kanokphon Sueby	67KaSu
67130500002	Kritsanaphong	71KrNg
67130500003	Kittikorn Phaen	13KiPh
67130500004	Kittithat Yotkh	30KiYo

> ตารางนี้คัดจากไฟล์ตัวอย่าง ไม่ได้ระบุว่าคุณเป็นนักศึกษาคนใด ตอนสอบจริงใช้ชื่อ รหัส และ Secret Key ของตัวเองตามใบงาน ห้ามใช้ข้อมูลของคนอื่น

สิ่งที่โจทย์ระบุชัด

ฐานข้อมูล mobile_distributor; ตั้งรหัสใน src/app.js; ทุก Route อยู่ใต้ /api/รหัสนักศึกษา; Server ใช้พอร์ต 3000; รายการสินค้าต้องแบ่งหน้าและส่ง Summary DTO; รายละเอียดส่ง Detail DTO; Status ต้องตรงแต่ละข้อ

จุดที่ชีทต้องเลือกวิธีทำเพิ่ม

เรื่อง	วิธีที่ใช้ในชีทนี้ ไม่ใช่เงื่อนไขเพิ่มเติมของข้อสอบ
Schema จริง	สมมติ tables brands / products และ columns ตามตัวอย่าง JSON; ต้องตรวจ SQL
ค่า limit เริ่มต้น	เลือก 10; spec ยก 10 เป็นตัวอย่าง แต่ไม่ได้ระบุ default ของ limit ชัดเจน
POST / PUT body ตอบกลับ	เลือกส่ง Detail DTO; spec ระบุสถานะ แต่ไม่ได้กำหนด JSON ตอบกลับสองข้อนี้
PUT request	รองรับทั้งส่งทุก field และบาง field; spec ไม่ระบุรูปแบบ body
Validation / Error JSON	เพิ่มตัวอย่าง 400 และรูปแบบ success:false เพื่อให้ debug ง่าย
timestamp	เลือกเวลาปัจจุบันรูปแบบ ISO; ไม่คัดลอกเวลาตัวอย่างตายตัว

คะแนนที่ระบุรายหัวข้อรวม 85 แต่ส่วน Self-Testing ระบุเต็ม 100 ให้ยึดชุดตรวจและคำชี้แจงผู้สอน ไม่เดาเพิ่มว่าอีก 15 คะแนนเป็นงานใด

ที่มา: [S1–S4] ส่วน Student Profile, §1–§4

00 / ทำก่อนข้อสอบทุกข้อ

ขั้นเตรียม 1: เปิดโฟลเดอร์และติดตั้ง

1. ต้องอยู่ในโฟลเดอร์ที่มี package.json จริง

จากภาพก่อนหน้า Terminal อยู่ใน __MACOSX และเห็น ._package.json จึงไม่ใช่รากโปรเจกต์ที่จะใช้ติดตั้ง อย่าเปลี่ยนชื่อ ._package.json มาแทน package.json

```
cd "$env:USERPROFILE\Downloads\node-express-esm-template"
Get-ChildItem -Recurse -Filter package.json |
  Where-Object { $_.FullName -notmatch '__MACOSX|node_modules' } |
  Select-Object -ExpandProperty DirectoryName
```

คัดลอก path ที่แสดง แล้วใช้ cd "path นั้น" จากนั้นเปิดโฟลเดอร์เดียวกันใน WebStorm ด้วย File → Open ไม่ใช่เปิด __MACOSX

```
Test-Path .\package.json
node -v
npm -v
npm install
```

Test-Path ต้องได้ True ก่อนสั่งติดตั้ง npm install จะติดตั้ง dependencies จาก package.json ไม่ต้องใช้ npm init สร้างโปรเจกต์ใหม่ทับชุดที่ได้รับ

2. เพิ่มคำสั่งเริ่ม Server โดยไม่ลบ dependencies เดิม

```
npm pkg set scripts.start="node src/app.js"
```

คำสั่งนี้เพิ่ม scripts.start ใน package.json ให้ npm start ไปเรียก src/app.js; ต้องคง "type": "module" เพื่อใช้ import/export ตาม Template

> อย่าเพิ่ง npm start จนกว่าจะสร้างไฟล์กลางและ app.js ถึงหน้า 8 เสร็จ ไม่เช่นนั้นอาจเจอ Cannot find module เพราะไฟล์ที่ import ยังไม่มี

ผ่านขั้นนี้เมื่อ

พบ package.json และ src ในโฟลเดอร์เดียวกัน, npm install จบโดยไม่มี npm error, มี node_modules และ package.json มีคำสั่ง start

ถ้า PowerShell แจ้งว่า npm.ps1 ถูกบล็อก ให้เรียก npm.cmd install และ npm.cmd start แทน โดยไม่ต้องเปลี่ยนนโยบายความปลอดภัยทั้งเครื่อง

ที่มา: [T] package.json; [W1] npm install / npm start; ภาพ Error ที่แนบในบทสนทนา

OO / SQL ก่อนเขียน Route

ขั้นเตรียม 2: นำเข้าและตรวจฐานข้อมูล

1. ตรวจว่ามีไฟล์ SQL ของข้อสอบจริง

```
Test-Path .\database\init-database.sql
```

> ZIP Template ที่แนบไม่มี database/init-database.sql และไม่มี tests/test-runner.mjs จึงยังใช้ไฟล์เหล่านี้ไม่ได้จนกว่าจะได้รับจากผู้สอน อย่าสร้างข้อมูลเดาแล้วถือว่าเป็นฐานข้อมูลข้อสอบจริง

2. เมื่อมี SQL แล้ว นำเข้าใน PowerShell

```
cmd /c "mysql -u root -p < database\init-database.sql"
```

ใส่รหัสผ่าน MySQL ของเครื่อง ไม่ใช่ Secret Key นักศึกษา ถ้าใช้ Command Prompt โดยตรงใช้ mysql -u root -p < database\init-database.sql ได้

3. เข้า MySQL แล้วตรวจชื่อจริง

```
mysql -u root -p
```

เมื่อ prompt เปลี่ยนเป็น mysql> จึงพิมพ์ SQL ต่อไปนี้ ไม่ใช่พิมพ์ SQL ที่ PS>

```
show databases;
use mobile_distributor;
show tables;
describe brands;
describe products;
show create table products;
select * from brands limit 3;
select * from products limit 3;
```

ต้องตรวจ	เพราะอะไร
id, sku, brand_id และชื่อ field อื่น	SQL / Mapper ต้องใช้ชื่อคอลัมน์จริง ไม่ใช่เดาจากชื่อ JSON
id สร้างอัตโนมัติหรือไม่	POST ในชีทใช้ result.insertId จาก auto_increment
sku มี UNIQUE หรือไม่	ต้องมีข้อบังคับระดับ DB เพื่อกันรายการซ้ำที่เข้าพร้อมกัน
release_date เป็น DATE หรือไม่	ชีทตั้ง dateStrings ให้ DATE กลับมาเป็น YYYY-MM-DD

ถ้าชื่อไม่ตรง ให้ปรับ SQL และ Mapper ให้เข้ากับ Schema จริง อย่าแก้ Schema ของข้อสอบโดยไม่มีคำสั่งจากผู้สอน

ที่มา: [S1–S4] §1.1; [T] src/configs/db-config.js

00 / แทนที่โค้ดในสองไฟล์นี้

ไฟล์กลาง 1: Database และ Response

1. แทนทั้งไฟล์ src/configs/db-config.js

```
import mysql from 'mysql2/promise';

const db = mysql.createPool({
  host: 'localhost',
  port: 3306,
  user: 'root',
  password: 'YOUR_MYSQL_PASSWORD',
  database: 'mobile_distributor',
  dateStrings: ['DATE']
});

export default db;
```

เปลี่ยน password ให้ตรงเครื่องจริง ไม่ใช้รหัสจาก Template โดยอัตโนมัติ createPool จัดการการเชื่อมต่อ; mysql2/promise ทำให้ใช้ await ได้; dateStrings: ['DATE'] คงค่า DATE เป็นข้อความ ไม่แปลงเป็นเวลา

2. แทนทั้งไฟล์ src/mappers/api-response.js

```
export function apiResponse(
  res, data, pagination = null, status = 200
) {
  const body = { success: true, data };

  if (pagination !== null) {
    body.pagination = pagination;
  }

  body.timestamp = new Date().toISOString();
  return res.status(status).json(body);
}
```

วิธีเรียก	ผลที่ได้
apiResponse(res, data)	200 พร้อม success, data, timestamp
apiResponse(res, data, pagination)	200 พร้อมข้อมูล pagination เพิ่ม
apiResponse(res, data, null, 201)	201 สำหรับเพิ่มสินค้าใหม่

> ลำดับพารามิเตอร์ในชีทนี้คือ res → data → pagination → status ค่า 201 ต้องเป็นตัวที่ 4 ห้ามเขียน apiResponse(res, data, 201) เพราะจะกลายเป็น pagination

ที่มา: [S1–S4] §1.1 และรูปแบบ response; [W3] mysql2; โค้ดตัวอย่างของชีท

OO / แทนทั้งไฟล์ src/middlewares/error-handler.js

ไฟล์กลาง 2: Error Handler

```
export class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
  }
}

export function errorHandler(err, req, res, next) {
  if (res.headersSent) return next(err);

  let status = err.statusCode || err.status || 500;
  let message = err.message || 'Unexpected error';

  if (err.code === 'ER_DUP_ENTRY') {
    status = 409;
    message = 'SKU or another unique value already exists';
  } else if (err.code === 'ER_NO_REFERENCED_ROW_2') {
    status = 400;
    message = 'Referenced record does not exist';
  } else if (err.code === 'ER_ROW_IS_REFERENCED_2') {
    status = 409;
    message = 'Record is still in use';
  }

  if (status >= 500) {
    console.error(err);
    message = 'Internal server error';
  }

  return res.status(status).json({
    success: false,
    error: { message },
    timestamp: new Date().toISOString()
  });
}
```

อ่านโค้ดนี้อย่างไร

throw new AppError('Product not found', 404) สร้าง Error พร้อมสถานะ แล้ว Express 5 ส่งไปที่ errorHandler; ต้องคงพารามิเตอร์ทั้ง 4 ตัว (err, req, res, next) และวาง app.use(errorHandler) หลัง Route ER_DUP_ENTRY ถูกแปลงเป็น 409 อีกชั้นหนึ่ง เผื่อ SKU ซ้ำตอน INSERT จริง ส่วน Error 500 แสดงรายละเอียดใน Terminal ไม่ส่งรายละเอียด SQL ทั้งหมดกลับให้ผู้ใช้

> โค้ด async ใน Route ชุดนี้อิง Express 5 ตาม Template หากผู้สอนให้ Express 4 ต้องเพิ่ม try/catch แล้วเรียก next(err) หรือใช้ async wrapper

ที่มา: [W2] Express 5 Error Handling; [T] error-handler.js; รูปแบบ Error เป็นตัวอย่างเพิ่ม

00 / ทำให้ app.js import ได้ครบ

ไฟล์กลาง 3: สร้างโครง Route ก่อน

1. สร้าง src/routes/brand-route.js แล้ววางทั้งบล็อก

```
import express from 'express';
import db from '../configs/db-config.js';
import { apiResponse } from '../mappers/api-response.js';
import { AppError } from '../middlewares/error-handler.js';

const router = express.Router();

// ADD BRAND ROUTES HERE

export default router;
```

2. แทนทั้งไฟล์ src/routes/product-route.js

```
import express from 'express';
import db from '../configs/db-config.js';
import { apiResponse } from '../mappers/api-response.js';
import { AppError } from '../middlewares/error-handler.js';
import * as mapper from '../mappers/product-mapper.js';

const router = express.Router();

// ADD HELPERS HERE

// ADD PRODUCT ROUTES HERE

export default router;
```

3. แทน src/mappers/product-mapper.js ด้วยบรรทัดนี้ก่อน

```
// Add the exported DTO functions from pages 12 and 15 here.
```

หน้า 12 และ 15 จะเพิ่มฟังก์ชันลงไฟล์ Mapper นี้ ส่วน import * as mapper หมายถึงอ่านฟังก์ชันที่ export จากไฟล์มาอยู่ใน object ชื่อ mapper

> ทุกบล็อก router.get/post/put/delete ที่เพิ่มภายหลังให้วางก่อน export default router; คง const router และ export อย่างละหนึ่งชุด อย่าคัดลอก scaffold ซ้ำทุกครั้ง

ชีทนี้เขียน SQL ใน Route เพื่อให้ตามขั้นได้ตรง ๆ ไฟล์ services / repositories / offices / productlines เดิมเก็บไว้ได้ แต่ app.js ชุดนี้ไม่เรียกใช้งาน และไม่ใช้ queryParser เก่าของ Template

ที่มา: [T] โครงสร้าง routes / mappers; โครงที่ลดความซับซ้อนสำหรับทำตาม

01 / 10 คะแนน / แทนทั้งไฟล์ src/app.js

ข้อ 2.1: Student Info — โค้ดเริ่ม Server

ทำหน้า 5–7 ให้ครบก่อน แล้ววางโค้ดนี้ เปลี่ยนตัวแปรข้อมูลนักศึกษา 3 ตัวตามหน้า 2 ตัวอย่างด้านล่างใช้ spec 00001 เท่านั้น

```
import express from 'express';
import brandRouter from './routes/brand-route.js';
import productRouter from './routes/product-route.js';
import { apiResponse } from './mappers/api-response.js';
import {
  AppError, errorHandler
} from './middlewares/error-handler.js';

const STUDENT_ID = '67130500001';
const STUDENT_NAME = 'Kanokphon Sueby';
const SECRET_KEY = '67KaSu';

const app = express();
app.use(express.json());

app.get(`/api/${STUDENT_ID}/student-info`, (req, res) => {
  return apiResponse(res, {
    studentId: STUDENT_ID,
    name: STUDENT_NAME,
    secretKey: SECRET_KEY
  });
});

app.use(`/api/${STUDENT_ID}/brands`, brandRouter);
app.use(`/api/${STUDENT_ID}/products`, productRouter);

app.use((req, res, next) => {
  next(new AppError('Endpoint not found', 404));
});
app.use(errorHandler);

app.listen(3000, () => {
  console.log(`API: http://localhost:3000/api/${STUDENT_ID}`);
});
```

> บันทึกทุกไฟล์ แล้วรัน npm start ในโฟลเดอร์ที่มี package.json ขณะนี้ Student Info ควรใช้ได้ ส่วน /brands และ /products จะยังเป็น 404 จนกว่าจะเพิ่ม Route ของแต่ละข้อ

ที่มา: [S1–S4] §1.2 และ §2.1; [W2] Express; โค้ดตัวอย่าง

01 / GET /student-info / ทำคู่กับหน้า 8

ข้อ 2.1: อธิบายและทดสอบ Student Info

ส่วนโค้ด	หน้าที่
const STUDENT_ID / STUDENT_NAME / SECRET_KEY	เก็บค่าจากใบงานเพียงจุดเดียว แล้วนำไปใส่ URL และ JSON
app.use(express.json())	เตรียมอ่าน JSON body ให้ POST / PUT; ต้องอยู่ก่อน Route
app.get(`/api/${STUDENT_ID}/student-info`, ...)	รับเฉพาะ GET ที่ URL นี้; backtick ใช้แทนค่าตัวแปรในข้อความ
apiResponse(res, {...})	ส่ง object นักศึกษาเป็น data และสถานะ default 200
app.use(..., brandRouter / productRouter)	เชื่อม Base Path เข้ากับ router ของแต่ละกลุ่ม
app.listen(3000, ...)	เริ่มฟัง request ที่พอร์ต 3000

ทดสอบใน Terminal อีกหน้าต่าง

```
$ID = '67130500001'
$BASE = "http://localhost:3000/api/$ID"
curl.exe -i "$BASE/student-info"
```

เปลี่ยน $ID ให้ตรง app.js ตัวอย่างโครง JSON ที่ควรได้มีดังนี้ โดย timestamp จะเป็นเวลาขณะตอบจริง

```
{
  "success": true,
  "data": {
    "studentId": "67130500001",
    "name": "Kanokphon Sueby",
    "secretKey": "67KaSu"
  },
  "timestamp": "2026-10-22T00:00:00.000Z"
}
```

ตรวจแล้วจึงไปข้อต่อไป

Status ต้องเป็น 200; studentId เป็น string; name และ secretKey ตรงทุกตัวอักษร; URL มีรหัสถูกชุด; ข้อนี้ไม่ต้อง query ฐานข้อมูล จึงใช้แยกตรวจได้ว่า Server และ Routing ทำงานแล้ว

> ถ้าใช้ GET / แล้วได้ 404 ยังไม่ได้แปลว่า Server เสีย เพราะเราไม่ได้สร้าง Route / ให้เรียก /api/รหัส/student-info ให้ครบ

ที่มา: [S1–S4] §2.1; [W2] Express request / response

02 / GET /brands / คะแนนรวม Brands 10

ข้อ 2.2-A: อ่านแบรนด์ทั้งหมด

1. เปิด src/routes/brand-route.js

เพิ่มบล็อกนี้ตรง // ADD BRAND ROUTES HERE และก่อน export default router โดยเก็บ import และ const router จากหน้า 7 ไว้

```
router.get('/', async (req, res) => {
  const [rows] = await db.query(
    'select * from brands order by id'
  );

  return apiResponse(res, rows);
});
```

2. เข้าใจทีละขั้น

router.get('/') ไม่ได้หมายถึงหน้าแรกทั้งเว็บไซต์ แต่เป็น / ภายใต้ Base Path ที่ app.js ผูกไว้ จึงได้ URL เต็ม /api/รหัส/brands

async ทำให้ใช้ await ได้; await db.query(...) รอผลจาก MySQL; select * เลือกทุกคอลัมน์ของ brands; order by id ทำให้เรียงผลตาม ID

mysql2 คืน [rows, fields] การเขียน const [rows] คือเลือกข้อมูลแถวออกมา ถ้าฐานข้อมูลมีหลายแบรนด์ rows จะเป็น Array; ถ้าไม่มีเลยจะเป็น []

apiResponse(res, rows) ส่งรายการทั้ง Array เป็น data ไม่ใช้ rows[0] เพราะข้อนี้ขอทุกแบรนด์

3. ทดสอบ

```
curl.exe -i "$BASE/brands"
```

ต้องได้ 200 และ data เป็น Array ของแบรนด์จริง เช่น แต่ละรายการมี id, name, country_of_origin, website, is_active, created_at ตามตัวอย่างโจทย์ ห้าม hardcode เฉพาะ Apple

ผิดแบบนี้ให้แก้ตรงไหน

อาการ	จุดตรวจ
500 และ Unknown database	นำเข้า SQL และเปลี่ยน database ใน db-config.js
500 และ Table does not exist	ตรวจ show tables; SQL ใช้ชื่อตารางตรงหรือไม่
ได้ object ไม่ใช่ Array	ตรวจว่าใช้ rows ไม่ใช่ rows[0]
ได้ 404 Endpoint not found	ตรวจ app.use(.../brands, brandRouter) และ restart Server

> ก่อนทำ Brand by ID ให้จำ ID ที่มีอยู่จริงจากผล /brands ไม่จำเป็นว่าทุกฐานข้อมูลต้องมี ID 1

ที่มา: [S1–S4] §2.2 รายการแบรนด์ทั้งหมด; [W3] mysql2 Promise Wrapper

03 / GET /brands/:id / ใช้ไฟล์เดียวกับหน้า 10

ข้อ 2.2-B: อ่านแบรนด์ตาม ID

1. เพิ่มใน src/routes/brand-route.js

วางต่อจาก GET /brands และก่อน export default router ห้ามลบ Route ที่เพิ่งทำผ่าน

```
router.get('/:id', async (req, res) => {
  const id = req.params.id;
  const [rows] = await db.query(
    'select * from brands where id = ?',
    [id]
  );

  if (rows.length === 0) {
    throw new AppError('Brand not found', 404);
  }

  return apiResponse(res, rows[0]);
});
```

2. อธิบายโค้ด

เรียก /brands/1 → req.params.id เป็นข้อความ "1"; where id = ? ใช้ค่าจาก [id] ไปแทน placeholder อย่างปลอดภัย ไม่ต่อข้อความ SQL ด้วยค่าจากผู้ใช้โดยตรง

แม้ค้นหาเพียงหนึ่ง ID ผล query ยังเป็น Array: พบจะได้ [object] และไม่พบจะได้ [] จึงตรวจ rows.length === 0 ก่อนเข้าถึง rows[0]

เมื่อพบส่ง rows[0] เป็น object หนึ่งตัว เมื่อไม่พบโยน AppError 404 ไม่ใช่ส่ง data:null พร้อม 200

3. ทดสอบทั้งสองทาง

```
curl.exe -i "$BASE/brands/1"
curl.exe -i "$BASE/brands/999999999"
```

บรรทัดแรกคาดหวัง 200 เฉพาะเมื่อ ID 1 มีจริง; บรรทัดที่สองคาดหวัง 404 เฉพาะเมื่อเลขนั้นไม่มีในฐานข้อมูล ถ้ามี ให้เลือกเลขที่ยืนยันแล้วว่าไม่มี

> จำให้แยก: รายการทั้งหมดส่ง rows; รายการเดียวส่ง rows[0]; หาไม่พบตรวจ rows.length ไม่ใช้ if (!rows) เพราะ [] ยังเป็นค่าที่ truthy

ที่มา: [S1–S4] §2.2 ข้อมูลแบรนด์ตามรหัส ID

04 / เริ่มข้อรายการสินค้า + Pagination / 30 คะแนน

ข้อ 2.3.1-A: สร้าง Summary DTO

1. เปิด src/mappers/product-mapper.js

เพิ่มฟังก์ชันนี้แทน comment ตั้งต้น โดยยังไม่ต้องสร้าง Route ใหม่ใน app.js เพราะ productRouter ถูกเชื่อมไว้แล้ว

```
export function toSummaryDTO(product) {
  return {
    id: product.id,
    sku: product.sku,
    name: product.name,
    brandId: product.brand_id,
    category: product.category,
    price: Number(product.price),
    status: product.status
  };
}
```

2. DTO คือเลือกและจัดรูปข้อมูลก่อนส่ง

ฐานข้อมูลอาจมี stock_quantity, ram_gb, storage_gb และคอลัมน์อื่น แต่รายการสินค้าตาม spec ต้องเหลือเฉพาะ 7 field ข้างบน ดังนั้นห้ามส่ง rows ดิบทั้งชุดโดยไม่แปลง

ข้อมูลจาก DB	JSON ที่ส่งออก	เหตุผล
brand_id	brandId	เปลี่ยนชื่อ snake_case เป็น camelCase; spec ยอมรับ brand_id ได้ด้วย
price เช่น "48900.00"	price: 48900	Number() แปลง DECIMAL ที่ driver อาจส่งมาเป็น string
stock_quantity	ไม่ส่งใน Summary	มีใน Detail ไม่ใช่ฟิลด์สรุปที่โจทย์กำหนด
id / sku / name / category / status	ใช้ชื่อเดิม	คงค่าจากแถวจริง ไม่ hardcode

3. ฟังก์ชันนี้ถูกใช้เมื่อใด

```
const data = rows.map(mapper.toSummaryDTO);
```

map วนแถวทีละสินค้า เรียก toSummaryDTO แล้วรวมผลเป็น Array ใหม่ เช่น มี rows 10 ตัว ก็ได้ data 10 ตัว แต่แต่ละตัวเหลือเฉพาะ field สรุป

> ใน product-route.js มี import * as mapper ไว้ตั้งแต่หน้า 7 จึงเรียก mapper.toSummaryDTO ได้ ตรวจสะกดชื่อให้ตรงและอย่าลืม export ที่หน้าฟังก์ชัน

ที่มา: [S1–S4] §2.3.1 ฟิลด์ Summary DTO; [W3] mysql2 DECIMAL

04 / GET /products?page=1&limit=10

ข้อ 2.3.1-B: โค้ดแบ่งหน้ารายการสินค้า

เพิ่มใน src/routes/product-route.js

วางใต้ // ADD PRODUCT ROUTES HERE และก่อน export default router ทำ Summary DTO หน้า 12 ก่อนแล้วจึงใช้โค้ดนี้

```
router.get('/', async (req, res) => {
  const page = Number(req.query.page ?? 1);
  const limit = Number(req.query.limit ?? 10);
  const offset = (page - 1) * limit;

  if (!Number.isSafeInteger(page) || page < 1 ||
      !Number.isSafeInteger(limit) || limit < 1 ||
      !Number.isSafeInteger(offset)) {
    throw new AppError('Invalid pagination', 400);
  }

  const [countRows] = await db.query(
    'select count(*) as total from products'
  );
  const total = Number(countRows[0].total);

  const [rows] = await db.query(
    'select * from products order by id limit ? offset ?',
    [limit, offset]
  );

  const data = rows.map(mapper.toSummaryDTO);
  const pagination = {
    total,
    page,
    limit,
    totalPages: Math.ceil(total / limit),
    hasNextPage: page * limit < total,
    hasPrevPage: page > 1
  };

  return apiResponse(res, data, pagination);
});
```

> COUNT ต้องนับสินค้าทั้งชุด ไม่ใช้ rows.length เป็น total เพราะ rows มีเฉพาะหน้าที่ดึงมา limit เริ่มต้น 10 และ 400 สำหรับ query ผิดเป็นนโยบายที่ชีทเพิ่ม ไม่ใช่ข้อกำหนดใหม่จาก spec

ที่มา: [S1–S4] §2.3.1; [W4] MySQL SELECT / LIMIT; โค้ดตัวอย่าง

04 / อธิบายโค้ดหน้า 13

ข้อ 2.3.1-C: คิด Pagination และทดสอบ

1. อ่าน page / limit จาก query string

URL /products?page=2&limit=10 ทำให้ req.query.page เป็น "2" และ req.query.limit เป็น "10" จึงใช้ Number(...) แปลงก่อนคำนวณ ส่วน ?? ใช้ค่าทางขวาเมื่อค่าทางซ้ายเป็น null หรือ undefined

2. สูตรที่ใช้ทุกครั้ง

```
offset = (page - 1) * limit;
totalPages = Math.ceil(total / limit);
hasNextPage = page * limit < total;
hasPrevPage = page > 1;
```

สมมติมี 75 รายการ	page	limit	offset	ผลที่ควรได้
หน้าแรก	1	10	0	10 ตัว / next:true / prev:false
หน้าที่สอง	2	10	10	10 ตัว / next:true / prev:true
หน้าสุดท้าย	8	10	70	5 ตัว / next:false / prev:true
เลยหน้าสุดท้าย	9	10	80	[] / next:false / prev:true

Math.ceil(75 / 10) ได้ 8 เพราะมีเศษก็ต้องเพิ่มอีกหนึ่งหน้า ถ้าฐานข้อมูลไม่มีสินค้า total=0 จะได้ totalPages=0; ชีทเลือกคืน [] พร้อม 200 เมื่อขอหน้าเกิน ไม่เปลี่ยน page ให้เอง

3. ทดสอบและอ่านค่าที่จำเป็น

```
curl.exe -i "$BASE/products"
curl.exe -i "$BASE/products?page=2&limit=5"
curl.exe -i "$BASE/products?page=0&limit=10"
curl.exe -i "$BASE/products?page=abc&limit=10"
```

สองคำสั่งแรกต้องได้ 200; จำนวน data ไม่เกิน limit; total ไม่เปลี่ยนตามหน้าเมื่อไม่มีการเพิ่ม/ลบสินค้า; totalPages ตรงสูตร; สองคำสั่งท้ายได้ 400 ตาม validation ที่เราเพิ่ม

> ใส่เครื่องหมาย "..." รอบ URL ที่มี & ใน Terminal เสมอ อย่าส่ง total:75 หรือ totalPages:8 ตายตัว เพราะ 75 ใน spec เป็นตัวอย่าง ไม่ใช่จำนวนแถวที่ยืนยันจาก SQL จริง

ที่มา: [S1–S4] §2.3.1; [W4] MySQL LIMIT; ตัวเลขต่อไปนี้เป็นตัวอย่างคำนวณ

05 / GET /products/:id / 10 คะแนน

ข้อ 2.3.2-A: สร้าง Detail DTO

เพิ่มใน src/mappers/product-mapper.js

วางต่อจาก toSummaryDTO หน้า 12 ไม่แทนที่ฟังก์ชันเดิม เพราะ GET รายการสินค้ายังต้องใช้ Summary DTO

```
export function toDetailDTO(product) {
  return {
    id: product.id,
    sku: product.sku,
    name: product.name,
    brandId: product.brand_id,
    brandName: product.brand_name,
    category: product.category,
    price: Number(product.price),
    ramGb: product.ram_gb,
    storageGb: product.storage_gb,
    color: product.color,
    stockQuantity: product.stock_quantity,
    status: product.status,
    releaseDate: product.release_date
  };
}
```

ข้อมูลที่เพิ่มจาก Summary DTO

JSON ของข้อสอบ	มาจากที่ใด
brandName	ชื่อจากตาราง brands; SQL ต้องตั้ง alias เป็น brand_name
ramGb / storageGb	products.ram_gb / products.storage_gb
stockQuantity	products.stock_quantity; คงค่า 0 ได้
color	products.color
releaseDate	products.release_date; เมื่อชนิด DATE จะเป็น YYYY-MM-DD ตาม db-config

> brandName จะไม่ปรากฏใน JSON หากค่าต้นทางเป็น undefined จึงต้องมี b.name as brand_name ใน SQL ของหน้าถัดไป ไม่ใช่ดึง products อย่างเดียว

ถ้า release_date ใน Schema จริงไม่ใช่ DATE ต้องปรับการแปลงให้ตรงรูปแบบวันที่ของโจทย์ อย่าใช้การตัด UTC โดยไม่ดูชนิดข้อมูลและเขตเวลา

ที่มา: [S1–S4] §2.3.2 Detail DTO; [W3] DATE และ DECIMAL

05 / เพิ่มสองบล็อกใน product-route.js

ข้อ 2.3.2-B: JOIN และค้นหาสินค้า

1. เพิ่ม helper ใต้ // ADD HELPERS HERE

```
async function findProduct(id) {
  const [rows] = await db.query(
    `select p.*, b.name as brand_name
     from products p
     left join brands b on p.brand_id = b.id
     where p.id = ?`,
    [id]
  );

  if (rows.length === 0) {
    throw new AppError('Product not found', 404);
  }

  return rows[0];
}
```

2. เพิ่ม Route ใต้ GET รายการสินค้า

```
router.get('/:id', async (req, res) => {
  const product = await findProduct(req.params.id);
  return apiResponse(res, mapper.toDetailDTO(product));
});
```

ทำไมต้องแยก findProduct

GET รายละเอียด, POST หลังเพิ่ม และ PUT ก่อน/หลังแก้ไข ต่างต้องค้นหาสินค้าตัวเดียวกัน จึงใช้ฟังก์ชันนี้ร่วมกัน แก้ SQL จุดเดียวก็มีผลกับทุกที่ที่เรียก

findProduct มีหน้าที่คืนแถวจากฐานข้อมูล ไม่ได้ส่ง response เอง ดังนั้น Route จึงนำผลไปผ่าน mapper.toDetailDTO แล้วส่งด้วย apiResponse อีกครั้ง

> ให้ helper อยู่ระดับเดียวกับ const router ไม่วางซ้อนไว้ภายใน router.get ของข้ออื่น และเก็บ export default router ไว้ท้ายไฟล์ตามเดิม

ที่มา: [S1–S4] §2.3.2; สมมติ FK products.brand_id → brands.id ตาม JSON

05 / อธิบายหน้า 15–16

ข้อ 2.3.2-C: อ่าน JOIN และทดสอบ

สมมติข้อมูลเล็ก ๆ เพื่อให้เห็นภาพ

products.id	products.name	products.brand_id
1	iPhone ตัวอย่าง	1
2	Galaxy ตัวอย่าง	2

brands.id	brands.name
1	Apple
2	Samsung

p เป็นชื่อย่อของ products และ b เป็นชื่อย่อของ brands; on p.brand_id = b.id จับคู่แบรนด์ของสินค้า; b.name as brand_name เปลี่ยนชื่อผลคอลัมน์เป็น brand_name เพื่อไม่ให้ชนกับ products.name

เลือก left join เพื่อยังได้แถวสินค้าหากชื่อแบรนด์จับคู่ไม่ได้ ในฐานข้อมูลที่มี Foreign Key ถูกต้อง กรณี brand_id อ้างถึงแถวที่ไม่มีควรถูกฐานข้อมูลป้องกันไว้ตั้งแต่เขียนข้อมูล

ทดสอบ

```
curl.exe -i "$BASE/products/1"
curl.exe -i "$BASE/products/999999999"
```

เลือก ID ที่มีจริงสำหรับคำสั่งแรก ต้องได้ 200, data เป็น object เดียว และมีฟิลด์ Detail DTO ครบ คำสั่งที่สอง เลือก ID ที่ไม่มี ต้องได้ 404

ตรวจ field ทีละจุด

เช็ก	ถ้าไม่ตรง
brandId เป็น ID, brandName เป็นชื่อ	ตรวจว่า map จาก brand_id และ brand_name คนละค่า
price เป็น JSON number	ตรวจ Number(product.price)
releaseDate เช่น 2023-09-22	ตรวจ SQL DATE และ dateStrings ใน db-config
ไม่มีฟิลด์ DB แปลก ๆ เพิ่ม	ตรวจว่าส่ง toDetailDTO ไม่ใช่ product ดิบ
ไม่พบ ID แล้วเป็น 404	ตรวจ rows.length ใน findProduct

> 404 ในข้อนี้คือไม่มีสินค้า แต่ 404 ข้อความ Endpoint not found มักเป็น URL หรือ Route ผิด อ่านข้อความ Error เพื่อแยกสาเหตุ

ที่มา: [S1–S4] §2.3.2; ชุดข้อมูลตารางด้านล่างสร้างขึ้นเพื่ออธิบาย

06 / POST /products / 10 คะแนน

ข้อ 2.3.3-A: เตรียมรับข้อมูล POST

เพิ่ม helper ใน src/routes/product-route.js

วางต่อจาก findProduct ในส่วน // ADD HELPERS HERE ใช้ร่วมกับ PUT ในหน้าถัดไป ไม่ต้องสร้างซ้ำ

```
const productFields = [
  'sku', 'name', 'brand_id', 'category', 'price',
  'ram_gb', 'storage_gb', 'color', 'stock_quantity',
  'status', 'release_date'
];

function readProductData(body) {
  if (!body || typeof body !== 'object' ||
      Array.isArray(body)) {
    throw new AppError('JSON object required', 400);
  }

  const data = {};
  for (const field of productFields) {
    if (Object.hasOwn(body, field)) {
      const value = body[field];
      if (value !== null &&
          typeof value !== 'string' &&
          typeof value !== 'number') {
        throw new AppError(`Invalid ${field}`, 400);
      }
      data[field] = value;
    }
  }
  return data;
}
```

สิ่งที่ helper ทำ

ตรวจว่า body เป็น JSON object แล้วเลือกเฉพาะ field ที่อยู่ใน productFields ไม่ยอมให้ส่ง id มาสั่งเปลี่ยน Primary Key และไม่เอา field แปลก ๆ ไปต่อเป็นชื่อคอลัมน์ SQL

Object.hasOwn(body, field) เช็กว่าผู้ใช้ส่ง key นี้มาจริงหรือไม่ จึงไม่ทิ้งค่า 0, "" หรือ null เพราะความเป็น false ของค่า ส่วนเงื่อนไขด้านในจำกัดค่าให้เป็น string, number หรือ null เพื่อไม่ส่ง object/array เข้า SQL

> นี่ไม่ใช่ validation ทุกเงื่อนไขธุรกิจ ยังต้องดู NOT NULL, ENUM และข้อจำกัดจริงใน SQL ผู้สอนไม่ได้ระบุครบใน spec ห้ามเดาค่า category / status ทั้งชุดเอง

ที่มา: [S1–S4] §2.3.3 Request Body; allow-list และ validation เป็นตัวอย่างเพิ่ม

06 / เพิ่มใต้ Route GET ใน product-route.js

ข้อ 2.3.3-B: โค้ดเพิ่มสินค้าและกัน SKU ซ้ำ

```
router.post('/', async (req, res) => {
  const data = readProductData(req.body);
  const required = ['sku', 'name', 'brand_id',
                    'category', 'price'];

  for (const field of required) {
    if (data[field] === undefined || data[field] === null) {
      throw new AppError(`Missing ${field}`, 400);
    }
  }

  const [duplicates] = await db.query(
    'select id from products where sku = ?', [data.sku]
  );
  if (duplicates.length > 0) {
    throw new AppError('SKU already exists', 409);
  }

  const fields = Object.keys(data);
  const values = Object.values(data);
  const marks = fields.map(() => '?').join(', ');

  const [result] = await db.query(
    `insert into products (${fields.join(', ')})
     values (${marks})`,
    values
  );

  const product = await findProduct(result.insertId);
  return apiResponse(res, mapper.toDetailDTO(product),
                     null, 201);
});
```

ก่อนคัดลอกต้องมีอะไรแล้ว

ต้องมี readProductData จากหน้า 18, findProduct จากหน้า 16 และ toDetailDTO จากหน้า 15; app.js ต้องใช้ express.json() ก่อนผูก Route

> required 5 field ในโค้ดนี้เป็น validation ขั้นต่ำที่ผู้จัดทำเลือก ไม่ใช่รายการ required ที่ spec ระบุ หาก SQL จริงมี NOT NULL เพิ่ม ให้ปรับตาม Schema และส่ง body ครบตามตัวอย่างโจทย์ก่อนทดสอบ

ที่มา: [S1–S4] §2.3.3: 201 เมื่อสร้าง, 409 เมื่อ SKU ซ้ำ

06 / สร้างหนึ่งครั้ง แล้วส่งซ้ำเพื่อเช็ก 409

ข้อ 2.3.3-C: อธิบาย POST และทดสอบ

อ่านการทำงานตามลำดับ

req.body → เลือก field → ตรวจข้อมูลขั้นต่ำ → หา SKU เดิม → สร้าง INSERT → รับ result.insertId → อ่านสินค้าที่เพิ่งเพิ่ม → ส่ง Detail DTO พร้อม 201

Object.keys(data) คือชื่อคอลัมน์; Object.values(data) คือค่าตามลำดับเดียวกัน; fields.map(() => '?') สร้าง placeholder เท่าจำนวนคอลัมน์ ตัวชื่อคอลัมน์มาจาก allow-list ส่วนค่าต้องผ่าน placeholder ไม่เอาค่ามาต่อ SQL

ทดสอบด้วย Postman หรือไฟล์ requests/api.http

เลือก POST, URL เป็น http://localhost:3000/api/รหัส/products, Body → raw → JSON และ Content-Type: application/json แล้วส่ง body นี้ ใช้ brand_id ที่มีอยู่จริง

```
{
  "sku": "TEST-SKU-999",
  "name": "Test Phone 2026",
  "brand_id": 1,
  "category": "FLAGSHIP",
  "price": 29900,
  "ram_gb": 8,
  "storage_gb": 256,
  "color": "Black",
  "stock_quantity": 10,
  "status": "AVAILABLE",
  "release_date": "2026-01-01"
}
```

ทดสอบ	ผลที่คาดหวัง
ส่งครั้งแรกเมื่อ SKU ยังไม่มี	201; จด data.id ที่ได้ไว้ใช้ PUT และ DELETE
ส่ง body เดิมอีกครั้งทันที	409; ไม่สร้างแถวใหม่ซ้ำ
GET /products/ID ที่เพิ่งสร้าง	200 และข้อมูลตรงกับที่ส่ง
ไม่ส่ง JSON หรือ body ผิดรูป	400 ตาม validation ในชีท

> หากเริ่มทดสอบแล้วได้ 409 ทันที อาจมี TEST-SKU-999 จากรอบก่อน เปลี่ยน SKU สำหรับรอบใหม่หรือจัดการเฉพาะรายการทดสอบ อย่าลบข้อมูลตั้งต้นของผู้สอน

ที่มา: [S1–S4] §2.3.3; โครง response หลังสร้างเป็นตัวอย่างของชีท

07 / PUT /products/:id / 10 คะแนน

ข้อ 2.3.4-A: โค้ดแก้ไขสินค้า

เพิ่มบล็อกนี้ใน src/routes/product-route.js ต่อจาก POST และก่อน export default router ใช้ helpers จากหน้า 16 และ 18 ชุดเดิม

```
router.put('/:id', async (req, res) => {
  const id = req.params.id;
  await findProduct(id);

  const data = readProductData(req.body);
  const fields = Object.keys(data);
  if (fields.length === 0) {
    throw new AppError('No fields to update', 400);
  }

  if (data.sku !== undefined) {
    const [duplicates] = await db.query(
      'select id from products where sku = ? and id <> ?',
      [data.sku, id]
    );
    if (duplicates.length > 0) {
      throw new AppError('SKU already exists', 409);
    }
  }

  const setClause = fields.map(f => `${f} = ?`).join(', ');
  const values = Object.values(data);

  await db.query(
    `update products set ${setClause} where id = ?`,
    [...values, id]
  );

  const product = await findProduct(id);
  return apiResponse(res, mapper.toDetailDTO(product));
});
```

> spec ไม่ระบุ PUT request body ชีทจึงรองรับการแก้ field ที่ส่งมา และรองรับ body ครบชุดด้วย ใน REST โดยทั่วไป การแก้บางส่วนมักใช้ PATCH แต่โจทย์นี้กำหนด Method เป็น PUT

ที่มา: [S1–S4] §2.3.4: 200 เมื่อแก้ไข, 404 เมื่อไม่พบ

07 / แก้เฉพาะสินค้าที่เพิ่ง POST

ข้อ 2.3.4-B: อธิบาย PUT และทดสอบ

ทำไมค้นหาก่อน UPDATE

await findProduct(id) ทำให้รู้ว่า ID มีจริงก่อนแก้ ถ้าไม่มีให้ 404 ได้ทันที ไม่ควรตัดสินว่าไม่มี ID จาก changedRows === 0 เพราะส่งค่าเดิมก็อาจไม่มีค่าที่เปลี่ยน ทั้งที่สินค้ายังอยู่

สร้าง SQL จากข้อมูลที่ส่งมาอย่างไร

```
// Example request body:
{ "price": 25900, "stock_quantity": 0 }

// Generated SQL:
update products
set price = ?, stock_quantity = ?
where id = ?;

// Example bound values when id = 80:
[25900, 0, "80"]
```

fields.map(f => `${f} = ?`).join(', ') สร้างส่วน SET; [...values, id] ต่อ ID ไว้หลังสุดเพื่อให้ตรง ? ใน where id = ?; ชื่อ field มาจาก allow-list เดิม ไม่ยอมรับชื่อคอลัมน์ตามอำเภอใจ

หากแก้ SKU ต้องไม่นับสินค้าตัวเองเป็นรายการซ้ำ จึงมี and id <> ?; ส่วนการส่ง Detail DTO หลัง UPDATE เป็นวิธีตอบกลับที่ชีทเลือก เพราะ spec บอกเพียงสถานะ

ทดสอบทีละกรณี

สิ่งที่ทำ	ผลที่ควรได้
PUT /products/ID-ใหม่ ด้วย body ตัวอย่างด้านบน	200; price เป็น 25900 และ stockQuantity เป็น 0
GET ID เดิมหลังแก้	200; ตรวจว่าฐานข้อมูลเปลี่ยนจริง
PUT body เดิมซ้ำอีกครั้ง	200 ไม่ใช่ 404 แม้ค่าทุกอย่างเหมือนเดิม
PUT ID ที่ยืนยันว่าไม่มี	404
PUT {} กับ ID ที่มี	400 ตามนโยบาย No fields to update
PUT SKU ของสินค้าอื่นที่มีอยู่	409 ตามการตรวจเพิ่มในตัวอย่าง

> ห้ามใช้ if (body.stock_quantity) เพื่อดูว่าผู้ใช้ส่งสต็อกหรือไม่ เพราะ 0 เป็น false; ใช้ Object.hasOwn หรือเปรียบเทียบ undefined ตามวัตถุประสงค์

ที่มา: [S1–S4] §2.3.4; รูปแบบ update และ 409 เพิ่มเติมเป็นนโยบายในตัวอย่าง

08 / DELETE /products/:id / 5 คะแนน

ข้อ 2.3.5: ลบสินค้า

1. เพิ่มท้าย Route ใน src/routes/product-route.js

วางก่อน export default router ไม่ใช้ apiResponse ในกรณีลบสำเร็จ เพราะโจทย์ต้องการไม่มี Response Body

```
router.delete('/:id', async (req, res) => {
  const [result] = await db.query(
    'delete from products where id = ?',
    [req.params.id]
  );

  if (result.affectedRows === 0) {
    throw new AppError('Product not found', 404);
  }

  return res.status(204).end();
});
```

2. เข้าใจผลจาก DELETE

ผล db.query สำหรับ DELETE ไม่ใช่ Array ของสินค้า แต่เป็นข้อมูลผลการทำงานอยู่ใน result; affectedRows บอกจำนวนแถวที่โดนลบ ถ้าเป็น 0 แปลว่าคำสั่งนี้ลบไม่พบแถวตาม ID

res.status(204).end() ตั้งสถานะแล้วจบ response โดยไม่ส่ง JSON การส่ง {success:true} หรือข้อความ "Deleted" ในข้อนี้ไม่ตรง spec

3. ทดสอบกับ ID ที่สร้างเองจาก POST

```
$NEWID = 80
curl.exe -i -X DELETE "$BASE/products/$NEWID"
curl.exe -i "$BASE/products/$NEWID"
curl.exe -i -X DELETE "$BASE/products/$NEWID"
```

แทน 80 ด้วย ID จริงจาก POST ของคุณ คำสั่งแรกต้องได้ 204 และไม่มี body; GET ถัดมาต้องได้ 404; DELETE ซ้ำต้องได้ 404 เช่นกัน

> อย่าเลือก ID 1 มาลบเพียงเพราะอยู่ในตัวอย่างโจทย์ ใช้เฉพาะรายการที่สร้างทดสอบเอง จะได้ไม่ทำข้อมูลตั้งต้นหายจนชุดตรวจข้ออื่นผิด

เมื่อทำหน้านี้เสร็จ

คุณมี Endpoint ครบ 8 รายการแล้ว: Student Info 1, Brands 2 และ Products 5 ให้ตรวจการประกอบไฟล์ในหน้าถัดไปก่อนรันชุดตรวจของผู้สอน

ที่มา: [S1–S4] §2.3.5: 204 ไม่มี body หรือ 404

09 / เช็กว่าแต่ละบล็อกอยู่ถูกที่

ประกอบไฟล์ให้ตรงก่อนทดสอบรวม

ไฟล์	ต้องมีอะไร	หน้าที่ทำ
src/app.js	ตั้งรหัส, Student Info, mount routers, error middleware, listen	8
src/configs/db-config.js	pool เชื่อม mobile_distributor	5
src/mappers/api-response.js	apiResponse(res,data,pagination,status)	5
src/middlewares/error-handler.js	AppError และ errorHandler	6
src/routes/brand-route.js	GET / และ GET /:id	7,10,11
src/mappers/product-mapper.js	toSummaryDTO และ toDetailDTO	12,15
src/routes/product-route.js	imports, router, helpers, GET/GET/POST/PUT/DELETE, export	7, 13, 16, 18, 19, 21, 23

ลำดับใน product-route.js

imports → const router → findProduct → productFields/readProductData → GET / → GET /:id → POST / → PUT /:id → DELETE /:id → export default router

> ใน ZIP ประกอบมี complete-project เป็นโค้ดสุดท้ายครบชุด ใช้เทียบกับสิ่งที่พิมพ์ ไม่ต้องนำไฟล์ .js ใน snippets มารันแยก เพราะบางบล็อกเป็นเพียงส่วนที่จะเพิ่มใน Route

ตรวจ Syntax โดยไม่ต้องรอฐานข้อมูล

```
Get-ChildItem .\src -Recurse -Filter *.js |
  ForEach-Object { node --check $_.FullName }
```

ถ้าไม่มีข้อความ Syntax Error แปลว่า parser อ่านโค้ดได้ ยังไม่ยืนยันว่า SQL, ID, ข้อมูล และสถานะตอบถูกต้องทดสอบ HTTP จริงต่อ

Restart หลังแก้ไฟล์

```
# In the server terminal: press Ctrl+C first.
npm start
```

Terminal นี้ต้องปล่อยให้ Server ทำงานอยู่ เปิดอีก Terminal สำหรับ curl และ test-runner อย่าเปิด npm start สองตัวใช้พอร์ต 3000 พร้อมกัน

ที่มา: โครงสร้างเฉลยในชีทนี้; ไม่ใช่ข้อบังคับให้แยกไฟล์ของ spec

09 / ผลที่ต้องเห็นก่อนส่งงาน

ตารางตรวจครบทุกข้อ

กำหนด $ID และ $BASE ตามหน้า 9 ใช้ ID ที่รู้ว่ามีจริงจาก GET รายการ และเลือกเลขที่ยืนยันว่าไม่มีสำหรับทดสอบ 404

การทดสอบ	สถานะ	ตรวจอะไรเพิ่มเติม
GET /student-info	200	ชื่อ รหัส Secret Key และ timestamp
GET /brands	200	data เป็น Array ข้อมูลจาก DB
GET /brands/ID-มี	200	data เป็น object เดียว
GET /brands/ID-ไม่มี	404	ไม่ตอบ success:true
GET /products	200	Summary 7 fields + pagination
GET /products?page=2&limit=5	200	page=2, limit=5, total นับทั้งชุด
GET /products/ID-มี	200	Detail ครบ รวม brandName และ releaseDate
GET /products/ID-ไม่มี	404	ไม่มีข้อมูลสินค้าตาม ID
POST /products ด้วย SKU ใหม่	201	จด data.id ของสินค้าทดสอบ
POST SKU เดิมอีกครั้ง	409	จำนวนแถวไม่เพิ่มซ้ำ
PUT /products/ID-ใหม่	200	GET ซ้ำแล้วข้อมูลเปลี่ยน
PUT /products/ID-ไม่มี	404	ไม่สร้างสินค้าใหม่แทน
DELETE /products/ID-ใหม่	204	ไม่มี body แม้แต่ JSON success
DELETE ID เดิมอีกครั้ง	404	ยืนยันลบไปแล้ว
เพิ่มเติม: page=0 หรือ page=abc	400	เป็น validation ที่เราเพิ่ม
เพิ่มเติม: PUT ค่าเดิม / stock=0	200	ไม่แปลค่าศูนย์ว่าไม่มีข้อมูล

ชุดทดสอบของผู้สอน

```
Test-Path .\tests\test-runner.mjs
node tests/test-runner.mjs 67130500001
```

รันบรรทัดที่สองเมื่อไฟล์มีจริง และเปลี่ยนรหัสให้ตรง spec โดย Server ต้องทำงานอยู่ก่อน ถ้าไม่พบไฟล์ ให้ขอชุดตรวจจริง ไม่ใช้ชุดตรวจที่แต่งขึ้นมารายงานเป็นคะแนนสอบ

ที่มา: [S1–S4] §2.1–§2.3.5; รายการท้ายตารางเป็น validation เพิ่มของชีท

09 / มีไฟล์ requests/api.http ให้ใช้ใน ZIP

วิธีส่ง Request ใน WebStorm / Postman

ทางเลือก A: WebStorm HTTP Client

เปิด requests/api.http ใน complete-project แล้วเปลี่ยน @base ให้ตรงรหัส จากนั้นกดปุ่ม Run ข้าง Request ทีละอัน ไม่กด Run All ก่อนแก้ @newId เพราะต้องได้ ID จาก POST ก่อน

```
@base = http://localhost:3000/api/67130500001
@newId = REPLACE_WITH_POST_ID

### Read products
GET {{base}}/products?page=1&limit=10

### Update only the product created for testing
PUT {{base}}/products/{{newId}}
Content-Type: application/json

{"price":25900,"stock_quantity":0}

### Delete the test product
DELETE {{base}}/products/{{newId}}
```

ไฟล์ใน ZIP มี POST พร้อม body ครบจากโจทย์ด้วย เมื่อ POST ผ่าน ให้คัดลอก data.id ไปแทน REPLACE_WITH_POST_ID แล้วจึงทดสอบ PUT และ DELETE

ทางเลือก B: Postman

ตั้งค่า	ใช้กับข้อใด
เลือก Method ให้ตรง GET / POST / PUT / DELETE	ทุกข้อ อย่าใช้ Browser ตรวจ POST
URL เริ่ม http://localhost:3000/api/รหัส	ทุกข้อ
Body → raw → JSON	POST และ PUT
Content-Type: application/json	POST และ PUT
อ่าน HTTP Status ไม่ใช่ดู JSON อย่างเดียว	ทุกข้อ โดยเฉพาะ 201,409,404,204

> URL ในชีทเป็นตัวอย่างให้แทนรหัสก่อนใช้งาน อย่าใส่ :id หรือคำว่า ID-ใหม่ ลงใน URL จริง ให้แทนด้วยเลขที่ได้จากระบบ

ที่มา: รูปแบบการทดสอบที่จัดทำเพิ่ม; JSON ของ POST มาจาก [S1–S4] §2.3.3

10 / ส่ง ZIP ที่ไม่มี node_modules

ตรวจและบีบอัดส่งข้อสอบ

ก่อนบีบอัด

รหัสใน app.js, URL, test-runner และชื่อไฟล์ ZIP ต้องตรงกัน; ทดสอบ Status ทุกกรณี; เก็บผลการตรวจจากชุดทดสอบจริง; เปลี่ยนรหัสผ่าน DB ตามคำแนะนำผู้สอน

วิธีทำผ่าน PowerShell จากรากโปรเจกต์

คำสั่งนี้คัดลอกเฉพาะไฟล์จำเป็นไปโฟลเดอร์ชั่วคราว จึงไม่ต้องลบ node_modules จากโปรเจกต์ที่กำลังทำงาน

```
$ID = '67130500001'
$stage = Join-Path $env:TEMP ("exam-" + [guid]::NewGuid())
New-Item -ItemType Directory -Path $stage | Out-Null

Copy-Item .\src $stage -Recurse
Copy-Item .\package.json, .\package-lock.json $stage

if (Test-Path .\database) {
  Copy-Item .\database $stage -Recurse
}
if (Test-Path .\tests) {
  Copy-Item .\tests $stage -Recurse
}

Compress-Archive -Path "$stage\*" `
  -DestinationPath ".\${ID}_exam.zip" -Force
```

เครื่องหมาย backtick ` ท้ายบรรทัด Compress-Archive คือบอก PowerShell ว่าคำสั่งยังต่อบรรทัดถัดไป ต้องไม่มีช่องว่างหลังเครื่องหมาย หรือเขียนสองบรรทัดสุดท้ายรวมเป็นบรรทัดเดียวก็ได้

เปิด ZIP ดูก่อนส่ง

ควรพบ src และ package.json / package-lock.json หากผู้สอนมี database, tests หรือไฟล์อื่นที่ต้องใช้ ให้แนบตามข้อกำหนดด้วย ตรวจว่าไม่มี node_modules หรือ __MACOSX ติดไป

> อย่าเลือกทั้งโฟลเดอร์ Downloads หรือทั้ง bundle ชีทไปส่ง ต้องส่งเฉพาะโปรเจกต์ข้อสอบที่ทำเสร็จ ไม่ใช่ PDF / snippets / mock tests ของชุดฝึก

ที่มา: [S1–S4] §3–§4 Submission Checklist

11 / ตารางแก้ Error ระหว่างทำ

ติดปัญหา เปิดหน้านี้ก่อน

ข้อความ / อาการ	สาเหตุที่ควรตรวจ	วิธีแก้
ENOENT package.json	อยู่ผิดโฟลเดอร์ / __MACOSX	กลับหน้า 3; Test-Path ต้อง True
npm.ps1 cannot be loaded	PowerShell บล็อก script	ลอง npm.cmd install / npm.cmd start
Missing script: start	package.json ไม่มี start	npm pkg set scripts.start="node src/app.js"
Cannot find module	ไฟล์ไม่ครบหรือ import path ผิด	ทำหน้า 5–7; ลงท้าย path ด้วย .js
does not provide an export	ชื่อ import/export ไม่ตรงกัน	เทียบชื่อ apiResponse, AppError และ mapper
EADDRINUSE :3000	Server อีกตัวใช้พอร์ตอยู่	Ctrl+C ตัวเก่าแล้วเริ่มใหม่ ไม่ย้ายพอร์ตมั่ว
mysql is not recognized	เครื่องหา mysql client ไม่เจอ	ใช้ Terminal ที่ตั้ง PATH ไว้ หรือ path ของ mysql.exe จริง
ER_ACCESS_DENIED_ERROR	MySQL user/password ไม่ตรง	แก้ db-config ไม่ใช้ Secret Key นักศึกษา
ECONNREFUSED	MySQL ไม่ทำงาน / host,port ผิด	ตรวจบริการ MySQL และค่าของเครื่องจริง
Unknown database / table	ยังไม่ import หรือชื่อผิด	ตรวจหน้า 4; อย่าเดาสร้าง DB แทน SQL จริง
ER_BAD_FIELD_ERROR	ชื่อคอลัมน์ผิด	describe products; แก้ SQL / Mapper ให้ตรง
Endpoint not found	URL หรือ Method ไม่ตรง Route	ตรวจ /api/รหัส และ GET/POST/PUT/DELETE
JSON object required	ไม่ส่ง JSON body หรือ Content-Type	เลือก raw JSON; ตรวจ express.json()
SKU already exists	POST ซ้ำกับข้อมูลรอบก่อน	ใช้ SKU ใหม่หรือจัดการเฉพาะแถวทดสอบ
price กลายเป็นข้อความ	ไม่แปลง DECIMAL ใน Mapper	ใช้ Number(product.price)
stockQuantity=0 ไม่ถูกบันทึก	เช็กค่าด้วย truthy	ใช้ hasOwn ไม่ใช้ if (body.stock_quantity)
DELETE มี body / สถานะผิด	ใช้ตัวช่วย JSON ส่งผล	ใช้ res.status(204).end()
แก้โค้ดแล้วผลไม่เปลี่ยน	ยังรัน Server เวอร์ชันก่อน	Save ทุกไฟล์แล้ว Ctrl+C / npm start

> แยกสาเหตุเป็นลำดับ: npm install ผ่านหรือยัง → Server เริ่มได้ไหม → student-info ผ่านไหม → brands ติดต่อ DB ได้ไหม → จึงตรวจ Products และ DTO

ที่มา: อ้างอิงภาพ Error ของผู้ใช้, โครง Template และพฤติกรรมของโค้ดในชีท

12 / ไม่ใช่ข้อที่ยืนยันว่าจะออก

แนวเสริม: ค้นหาสินค้าด้วยชื่อ

สมมติโจทย์ใหม่เพิ่ม GET /products?q=phone จุดที่ต้องแก้คือ Route รายการสินค้าเดิม ห้ามสร้าง GET / ซ้ำอีกตัวแล้วคาดว่าทั้งสองตัวจะทำงานต่อกันเอง

แก้ส่วน COUNT และดึงแถวให้ใช้เงื่อนไขเดียวกัน

โค้ดด้านล่างเป็นส่วนแทน COUNT/SELECT ภายใน Route หน้า 13 โดยคง page, limit, offset, DTO และการสร้าง pagination เดิมไว้

```
const q = String(req.query.q ?? '').trim();
const where = q ? 'where name like ?' : '';
const params = q ? [`%${q}%`] : [];

const [countRows] = await db.query(
  `select count(*) as total from products ${where}`,
  params
);
const total = Number(countRows[0].total);

const [rows] = await db.query(
  `select * from products ${where}
   order by id limit ? offset ?`,
  [...params, limit, offset]
);
```

LIKE ใช้ % เพื่อจับข้อความบางส่วน ชื่อ field และส่วน SQL มาจากโค้ดที่กำหนดเอง ค่า q ต้องส่งใน params; total ต้องนับเฉพาะสินค้าที่ตรงคำค้น ไม่ใช่นับทั้งตารางก่อนกรอง

รูปแบบโจทย์ใกล้เคียงที่ฝึกต่อได้

โจทย์ดัดแปลง	แนวแก้
กรอง brand_id / status	เพิ่ม where จากเงื่อนไขที่อนุญาต และใช้ params ใน COUNT กับ SELECT เหมือนกัน
เรียง price มากไปน้อย	กำหนด allow-list เช่น id/name/price; ห้ามต่อชื่อคอลัมน์จาก input ดิบ
GET /products/stats	วาง Route คำคงที่ /stats ก่อน /:id เพื่อไม่ให้ stats ถูกอ่านเป็น ID
GET /brands/:id/products	กรอง products.brand_id ตาม req.params.id และทำ DTO ตามโจทย์ใหม่

> ไม่มีหลักฐานจากไฟล์ที่ให้มาว่าแนวเหล่านี้จะออกหรือมีโอกาสกี่เปอร์เซ็นต์ ฝึกหลังทำ Endpoint ที่ระบุใน spec ครบก่อน

ที่มา: ตัวอย่างต่อยอดของผู้จัดทำ ไม่ได้ระบุใน spec ทั้งสี่ชุด

12 / แยกจากข้อกำหนดหลัก

แนวเสริม: Validation และคำถามกับดัก

ตัวอย่าง: ถ้าโจทย์เพิ่มว่าสต็อกต้องเป็นจำนวนเต็มตั้งแต่ 0

เพิ่มหลัง readProductData ใน POST/PUT เมื่อโจทย์กำหนดเงื่อนไขนี้จริง โค้ดตรวจเฉพาะตอนส่ง field มา จึงไม่บังคับ PUT ให้ส่งทุกคอลัมน์

```
if (Object.hasOwn(data, 'stock_quantity')) {
  if (!Number.isInteger(data.stock_quantity) ||
      data.stock_quantity < 0) {
    throw new AppError('Invalid stock_quantity', 400);
  }
}
```

ตัวอย่าง: ถ้าโจทย์กำหนดให้ราคาต้องเป็น number ไม่ติดลบ

```
if (Object.hasOwn(data, 'price')) {
  if (typeof data.price !== 'number' ||
      !Number.isFinite(data.price) || data.price < 0) {
    throw new AppError('Invalid price', 400);
  }
}
```

คำถาม / input	คำตอบที่ควรเข้าใจ
stock_quantity: 0	0 ไม่ใช่ข้อมูลหาย ต้องผ่านเงื่อนไขจำนวนเต็มไม่ติดลบ
stock_quantity: "10"	เป็น string ไม่ใช่ number; ตัวอย่าง validation นี้ไม่แปลงให้เอง
stock_quantity: null	ไม่ใช่จำนวนเต็ม; จะยอมรับหรือไม่ขึ้นกับโจทย์/Schema
PUT ค่าเดิมทั้งหมด	ยังพบ resource อยู่ จึงไม่ควรตอบ 404
ลบครั้งที่สอง	แถวหายแล้ว จึงได้ 404
SELECT ได้ []	เป็น Array ว่างที่ truthy ต้องเช็ก length
มี 21 รายการ limit=10	ต้องมี 3 หน้า ไม่ใช่ 2 เพราะหน้าสุดท้ายมี 1 รายการ

> อย่าเพิ่มกฎที่คิดเองจน request ที่ถูกตาม spec ถูกปฏิเสธ เช่น ตั้ง limit สูงสุด 100 หรือเดาค่า ENUM ทั้งหมด ทั้งที่ผู้สอนไม่ได้กำหนด

ที่มา: ตัวอย่างฝึกเพิ่มของผู้จัดทำ; ต้องใช้ชนิด/ช่วงค่าตามโจทย์จริง

13 / เปิดอ้างอิงระหว่างเขียน

ชีทจำเป็น: อ่านโค้ดให้ทันในห้องสอบ

สิ่งที่ต้องจำ	ความหมาย / ตัวอย่าง
req.params.id	ค่าจาก /products/5 → "5"
req.query.page	ค่าจาก ?page=2 → "2"; แปลงด้วย Number ก่อนคำนวณ
req.body	JSON ที่ส่งมากับ POST/PUT; ต้องมี express.json()
const [rows] = await db.query(...)	หยิบส่วนข้อมูลแถวจากผลลัพธ์ [rows, fields]
rows[0]	object แรกใน Array; ใช้กับรายละเอียดหนึ่งรายการ
rows.map(mapper.toSummaryDTO)	แปลงทุกสินค้าเป็นรูปแบบสรุป
Object.keys / Object.values	ชื่อ key / ค่าของ object ตามลำดับที่สอดคล้องกัน
Object.hasOwn(body, field)	มี key นี้ถูกส่งมาจริงไหม ไม่ตัดสินจาก truthy ของค่า
??	ใช้ค่า default เมื่อซ้ายเป็น null หรือ undefined
return	จบฟังก์ชันนี้ ป้องกันโค้ดข้างล่างทำงานต่อโดยไม่ตั้งใจ
throw new AppError(message,status)	ส่งเรื่องให้ Error Handler จัด HTTP error
result.insertId	ID ที่ฐานข้อมูลสร้างให้รายการ INSERT
result.affectedRows	จำนวนแถวที่คำสั่งเขียนมีผล; DELETE ใช้ตรวจว่าลบพบไหม
export default router	ให้ app.js import router ของไฟล์นี้ไปเชื่อม URL

Status ที่ใช้บ่อยในชีท

200	201	204	400	404	409	500
อ่าน/แก้สำเร็จ	สร้างสำเร็จ	สำเร็จ ไม่มี body	ข้อมูล request ผิด	ไม่พบ	ชนข้อมูลเดิม	ข้อผิดพลาด Server

คำสั่ง SQL คู่กับ HTTP

```
GET     -> select ... from ... where ...
POST    -> insert into ... (...) values (...)
PUT     -> update ... set ... where id = ?
DELETE  -> delete from ... where id = ?
```

> UPDATE และ DELETE ต้องมี where ที่ถูกต้องเสมอ อย่าลบ where เพื่อทดลอง เพราะจะกระทบหลายแถวหรือทั้งตาราง

ที่มา: [S1–S4] รูปแบบ API; [W2–W4] คำอธิบายเทคนิค

14 / อ่านก่อนสรุปว่าผ่านข้อสอบ

แหล่งที่มาและขอบเขตของชีท

ฐานเนื้อหาที่ผู้ใช้แนบ

[S1] exam-spec-67130500001(1).md — Student Profile, §1 Setup, §2 Endpoints, §3 Self-Testing, §4 Submission

[S2] exam-spec-67130500002(1).md — โจทย์ชุดเดียวกัน โดยข้อมูลนักศึกษาตามหน้า 2

[S3] exam-spec-67130500003(1).md — โจทย์ชุดเดียวกัน โดยข้อมูลนักศึกษาตามหน้า 2

[S4] exam-spec-67130500004(1).md — โจทย์ชุดเดียวกัน โดยข้อมูลนักศึกษาตามหน้า 2

[T] node-express-esm-template.zip — ตรวจโครงสร้างจริง พบ express ^5.2.1, mysql2 ^3.24.3, type:module, ฐานข้อมูลเดิม classicmodels และยังไม่มี start script; ไม่พบ SQL และ test-runner ที่ใบงานอ้างถึง

เอกสารทางการที่ใช้ตรวจหลักการเขียนโค้ด

[W1] npm Docs: npm install, npm start และ package.json — การติดตั้ง dependencies และ scripts

[W2] Express 5: API Reference และ Error Handling — Router, JSON middleware, response และ async error

[W3] MySQL2: Quickstart, Promise Wrapper, Documentation และ ConnectionOptions — query result, DECIMAL และ dateStrings

[W4] MySQL 8.4 Reference Manual: SELECT Statement — ORDER BY, LIMIT และ OFFSET

ตรวจแล้ว / ยังไม่ได้ยืนยัน

ตรวจ Syntax ของ JavaScript 7 ไฟล์ด้วย node --check และทดสอบตรรกะ Route/DTO ผ่าน 38 กรณีด้วย Express/ฐานข้อมูลจำลอง (mock); ไม่ใช่การยิง HTTP จริง ไม่ได้ทดสอบ MySQL จริง และไม่ใช่ชุดตรวจของผู้สอน

> ไม่มี SQL Schema จริงและ tests/test-runner.mjs ของผู้สอน จึงไม่อ้างว่าผ่านคะแนนตรวจอัตโนมัติหรือเชื่อม MySQL จริงได้ทุกเครื่อง ต้องทดสอบกับชุดจริงก่อนส่ง

สิ่งที่ออกแบบเพิ่ม ไม่ใช่ข้อความจากใบงาน

โค้ดเฉลยและ helpers, Error JSON, validation, default limit=10, รูปแบบ PUT, POST/PUT ตอบ Detail DTO, การทดสอบ และแนวเสริมหน้า 29–30 เป็นวิธีทำที่จัดทำเพิ่มเพื่ออธิบาย ไม่ใช่กติกาเพิ่มเติมจากผู้สอน

วันที่จัดทำ: 22 ตุลาคม 2026 • ฉบับทำตามทีละข้อ • ใช้ข้อมูลประจำตัวใน spec ของตนเองเท่านั้น